

Universidad del Valle de Guatemala
Facultad de Ingeniería
Programación Orientada a Objetos
Segundo Ciclo, 2026
Catedrático: Erick Marroquín



Ejercicio 3

Arreglos dinámicos, ArrayList, List y Excepciones

Nombre: Laura Suseth Morales Murga
Carné: 26281
Sección # 30
Fecha de entrega: 25/08/26

Análisis

Responda las siguientes preguntas:

Tabla No. 1: Tabla de propiedades

No.	Tipo de datos	Nombre variable	Visibilidad	Propósito	Clase
1	int	numeroOrden	Private	Almacenar el número único que identifica a cada orden de servicio.	OrdenServicio
2	String	propietario	Private	Almacenar el nombre del propietario del vehículo asociado a la orden.	OrdenServicio
3	String	placa	Private	Almacenar la placa del vehículo asociado a la orden de servicio.	OrdenServicio
4	String	descripcionServicio	Private	Almacenar la descripción del mantenimiento o reparación solicitado.	OrdenServicio
5	double	costoEstimado	Private	Almacenar el costo estimado de la orden de servicio, el cual deberá ser mayor que cero.	OrdenServicio
6	List<OrdenServicio>	ordenes	Private	Almacenar dinámicamente las órdenes de servicio registradas. La colección será inicializada utilizando un ArrayList<OrdenServicio>.	GestorOrdenes
7	Scanner	scanner	Private	Leer los datos ingresados por el usuario durante la interacción con el sistema.	TallerVista

8	GestorOrdenes	gestor	Private	Mantener una referencia al modelo para solicitar las operaciones relacionadas con las órdenes de servicio.	TallerControlador
9	TallerVista	vista	Private	Mantener una referencia a la vista para solicitar datos al usuario y mostrar los resultados de las operaciones.	TallerControlador

Tabla No. 2: Tabla de métodos

No	Tipo de retorno	Nombre	Visibilidad	Parámetros	Propósito	Clase
1	Constructor	OrdenServicio	Public	int numeroOrden, String propietario, String placa, String descripcionServicio, double costoEstimado	Inicializar una orden de servicio con la información proporcionada.	OrdenServicio
2	int	getNumeroOrden	Public	Ninguno	Obtener el número que identifica a la orden de servicio.	OrdenServicio
3	String	getPropietario	Public	Ninguno	Obtener el nombre del propietario del vehículo.	OrdenServicio
4	String	getPlaca	Public	Ninguno	Obtener la placa del vehículo asociado a la orden.	OrdenServicio
5	String	getDescripcionServicio	Public	Ninguno	Obtener la descripción del servicio solicitado.	OrdenServicio
6	double	getCostoEstimado	Public	Ninguno	Obtener el costo estimado de la orden de servicio.	OrdenServicio
7	void	setDescripcionServicio	Public	String descripcionServicio	Modificar la descripción del servicio de una orden existente.	OrdenServicio
8	void	setCostoEstimado	Public	double costoEstimado	Modificar el costo estimado de una orden existente.	OrdenServicio
9	String	toString	Public	Ninguno	Retornar en texto toda la información correspondient	OrdenServicio

					e a la orden de servicio.	
10	Constructor	GestorOrdenes	Public	Ninguno	Inicializar el gestor y crear la colección dinámica utilizando un ArrayList.	GestorOrdenes
11	void	agregarOrden	Public	OrdenServicio orden	Validar y agregar una nueva orden de servicio a la colección dinámica.	GestorOrdenes
12	List<OrdenServicio>	obtenerOrdenes	Public	Ninguno	Obtener las órdenes de servicio almacenadas actualmente en la colección.	GestorOrdenes
13	OrdenServicio	buscarOrden	Public	int numeroOrden	Buscar una orden mediante su número y retornarla. Si no existe, controlar la operación mediante una excepción.	GestorOrdenes
14	void	modificarOrden	Public	int numeroOrden, String descripcionServicio, double costoEstimado	Modificar la descripción y el costo estimado de una orden previamente registrada.	GestorOrdenes
15	void	cancelarOrden	Public	int numeroOrden	Buscar una orden mediante su número y eliminarla de la colección dinámica.	GestorOrdenes
16	List<OrdenServicio>	buscarPorPlaca	Public	String placa	Buscar y retornar todas las órdenes asociadas a una	GestorOrdenes

					placa específica.	
17	double	calcularTotal	Public	Ninguno	Recorrer las órdenes activas y calcular la suma de sus costos estimados.	GestorOrdenes
18	double	calcularPromedio	Public	Ninguno	Calcular el costo promedio de las órdenes registradas actualmente.	GestorOrdenes
19	OrdenServicio	obtenerOrdenMayorCosto	Public	Ninguno	Recorrer la colección y retornar la orden que posea el costo estimado más alto.	GestorOrdenes
20	int	cantidadOrdenes	Public	Ninguno	Retornar la cantidad de órdenes de servicio almacenadas actualmente.	GestorOrdenes
21	Constructor	TallerVista	Public	Ninguno	Inicializar la vista y el Scanner utilizado para recibir información del usuario.	TallerVista
22	void	mostrarMenu	Public	Ninguno	Mostrar las diez opciones disponibles en el menú principal del sistema.	TallerVista
23	String	solicitarDato	Public	String mensaje	Mostrar un mensaje y solicitar al usuario un dato de tipo texto.	TallerVista
24	int	solicitarEntero	Public	String mensaje	Solicitar un dato al usuario	TallerVista

					y convertirlo a un valor entero.	
25	double	solicitarDouble	Public	String mensaje	Solicitar un dato al usuario y convertirlo a un valor decimal.	TallerVista
26	void	mostrarMensaje	Public	String mensaje	Mostrar al usuario un mensaje generado durante la ejecución del sistema.	TallerVista
27	void	mostrarOrden	Public	OrdenServicio orden	Mostrar toda la información correspondiente a una orden de servicio.	TallerVista
28	void	mostrarOrdenes	Public	List<OrdenServicio> ordenes	Recorrer y mostrar una lista de órdenes de servicio.	TallerVista
29	Constructor	TallerControlador	Public	GestorOrdenes gestor, TallerVista vista	Inicializar el controlador con las referencias al modelo y a la vista que utilizará durante la ejecución.	TallerControlador
30	void	iniciar	Public	Ninguno	Mantener en ejecución el menú principal y coordinar la opción seleccionada hasta que el usuario decida salir.	TallerControlador
31	void	registrarOrden	Private	Ninguno	Solicitar mediante la vista los datos necesarios y coordinar el	TallerControlador

					registro de una nueva orden.	
32	void	consultarOrdenes	Private	Ninguno	Obtener del modelo las órdenes registradas y enviarlas a la vista para mostrarlas.	TallerControlador
33	void	buscarOrden	Private	Ninguno	Solicitar el número de una orden, realizar su búsqueda y mostrar el resultado o controlar la excepción correspondiente.	TallerControlador
34	void	modificarOrden	Private	Ninguno	Solicitar los nuevos datos de una orden y coordinar su modificación mediante el modelo.	TallerControlador
35	void	cancelarOrden	Private	Ninguno	Solicitar el número de una orden y coordinar su eliminación de la colección.	TallerControlador
36	void	consultarPorPlaca	Private	Ninguno	Solicitar una placa y coordinar la búsqueda y visualización de todas las órdenes asociadas.	TallerControlador
37	void	mostrarReporteCostos	Private	Ninguno	Obtener del modelo el costo total y promedio de las órdenes y mostrarlos	TallerControlador

					mediante la vista.	
38	void	mostrarOrdenMayorCosto	Private	Ninguno	Obtener del modelo la orden con el costo estimado más alto y mostrar su información.	TallerControlador
39	void	mostrarCantidadOrdenes	Private	Ninguno	Obtener del modelo y mostrar la cantidad actual de órdenes registradas.	TallerControlador
40	void	main	Public Static	String[] args	Crear las instancias del modelo, vista y controlador e iniciar la ejecución del sistema.	Main

1. ¿Qué clases son necesarias para representar el problema planteado?
 - a. Usaré las clases OrdenServicio, GestorOrdenes, TallerVista, TallerControlador y Main.
2. ¿Qué propiedades y métodos tendrá cada clase?
 - a. Las propiedades que utilizaré en cada clase se encuentran detalladas en la Tabla No. 1: Tabla de propiedades, mientras que los métodos, constructores, getters y setters se encuentran en la Tabla No. 2: Tabla de métodos.
3. ¿Qué tipo deben tener las propiedades y métodos de cada clase?
 - a. Los tipos de datos de cada propiedad y los tipos de retorno de los métodos están especificados en la Tabla No. 1 y la Tabla No. 2.
4. ¿Qué información deberá almacenarse dentro de la colección dinámica?
 - a. Dentro de la colección dinámica almacenaré objetos de tipo OrdenServicio. Cada objeto contendrá el número de orden, nombre del propietario, placa del vehículo, descripción del servicio y costo estimado.
 - b. La colección va a ser así: `List<OrdenServicio>` Así se puede almacenar una cantidad variable de órdenes sin tener un límite fijo.
5. ¿Dónde utilizará List y dónde realizará la creación mediante ArrayList?
 - a. Usaré `List<OrdenServicio>` como tipo de la propiedad ordenes dentro de la clase GestorOrdenes, tal como se indica en la Tabla No. 1.
 - b. La creación de la colección se realizará en el constructor de GestorOrdenes utilizando un ArrayList: `ordenes = new ArrayList<OrdenServicio>();` Así voy a usar List como interfaz para declarar la colección y ArrayList como su implementación ya para el programa.
6. ¿Cuáles deben ser los modificadores de visibilidad de los miembros de cada clase?
 - a. Los modificadores específicos se encuentran representados en las Tablas No. 1 y No. 2.

7. ¿Qué parámetros serán requeridos por los métodos de sus clases?
 - a. Los parámetros de cada método se encuentran especificados en la Tabla No. 2: Tabla de métodos.
8. ¿Cómo proveerá de valores iniciales a sus objetos? ¿Qué valores iniciales les asignará?
 - a. Pondré los valores iniciales de cada OrdenServicio con su constructor, usando la información del usuario. Así una orden será creada desde el inicio con su número, propietario, placa, descripción y costo estimado.
 - b. En GestorOrdenes, inicializaré la colección como un ArrayList vacío, ya que cuando inicie el programa todavía no tendrá órdenes registradas. TallerVista inicializará su Scanner, y en TallerControlador recibirá mediante su constructor las instancias de GestorOrdenes y TallerVista.
9. ¿Cómo identificará una orden específica dentro de la colección?
 - a. Identificaré cada orden mediante la propiedad numeroOrden, que va a funcionar como un identificador. Para buscar una orden voy a recorrer la colección y voy a comparar el número que se quiere buscar con el número de cada objeto.
 - b. Antes de registrar una nueva orden también voy a comprobar que su número no la tenga otra orden.
10. ¿Cómo agregará y eliminará órdenes de la colección dinámica?
 - a. Para agregar una orden utilizaré el método add() de la colección después de comprobar que los datos sean válidos y que el número de orden no esté registrado de antes.
 - b. Para eliminar una orden, primero la voy a buscar por su número y usaré remove() sobre la colección. Esto lo voy a hacer en el método cancelarOrden de GestorOrdenes.
11. ¿Cómo realizará la búsqueda de todas las órdenes asociadas a una misma placa?
 - a. Voy a recorrer la colección completa y compararé la placa ingresada por el usuario con la propiedad placa de cada OrdenServicio.
 - b. Cada coincidencia será agregada a una nueva List<OrdenServicio>, la cual será retornada al finalizar la búsqueda. De esta manera podré mostrar todas las órdenes asociadas a una misma placa y no solamente la primera coincidencia encontrada.

12. ¿Cómo recorrerá la colección para calcular el valor total y el costo promedio de las órdenes?
- Recorreré la colección de órdenes y utilizaré una variable acumuladora para sumar el costoEstimado de cada objeto. El resultado de esta suma será el valor total de las órdenes activas.
 - Para calcular el promedio dividiré el costo total entre la cantidad de órdenes almacenadas. Antes de realizar la división verificaré que exista al menos una orden para evitar realizar una división entre cero.
13. ¿Cómo determinará cuál es la orden con el costo estimado más alto?
- Recorreré la colección comparando el costoEstimado de las órdenes. Mantendré una referencia a la orden que tenga el costo mayor encontrado hasta ese momento y la actualizaré cuando encuentre una orden con un costo superior.
 - Al terminar el recorrido, retornaré el objeto OrdenServicio correspondiente al costo estimado más alto.
14. ¿Qué situaciones dentro del programa pueden producir una excepción?
- Pueden producirse excepciones cuando el usuario ingrese un tipo de dato incorrecto, por ejemplo, texto cuando se espera recibir el número de una orden o un costo estimado. También pueden ocurrir al intentar buscar, modificar o cancelar una orden que no se encuentre registrada.
 - Además, consideraré como situaciones inválidas el ingreso de un propietario, placa o descripción vacíos, un costo estimado menor o igual a cero y el intento de registrar un número de orden que ya exista.
15. ¿Qué operaciones serán protegidas utilizando try-catch?
- Utilizaré try-catch para proteger las operaciones que procesen entradas numéricas del usuario y aquellas que dependan de encontrar una orden registrada.
 - Utilizaré NumberFormatException para controlar los casos en los que una entrada no pueda convertirse correctamente a un valor numérico. Para operaciones como buscar, modificar o cancelar una orden inexistente, utilizaré IllegalArgumentException, permitiendo mostrar un mensaje al usuario y regresar posteriormente al menú principal sin finalizar inesperadamente el programa.

16. ¿Qué acción realizará mediante el bloque finally y por qué debe ejecutarse independientemente del resultado de la operación?
- a. Utilizaré un bloque finally durante el proceso de búsqueda de una orden. Dentro de este bloque mostraré un mensaje indicando que el proceso de búsqueda ha finalizado.
 - b. Este mensaje se mostrará independientemente de que la orden haya sido encontrada correctamente o se haya producido una excepción. De esta manera utilizaré finally para realizar una acción que debe ejecutarse sin importar el resultado de la operación..